Design and apply context-aware access controls based on user identity, device posture, location, and application risk

College Name: Vidyasagar college of arts and science College

Code: BRU30

Team ID: SWTID-2026-4971

Team Leader : RAJAMANICKAM.C

Email: rajamanickamc231044@gmail.com

Team Member: MANJUNATH.S

Email: manjunaths231036@gmail.com

Team Member: KALIMUTHU.P

Email: kalimuthu231031@gmail.com

Team Member: GOPAL.R

Email: gopalr231026@gmail.com

Team Member: HARINI G

Email: harinig231029@gmail.com

Stage – 1

Title

Design and Apply Context-Aware Access Controls Based on User Identity, Device Posture, Location, and Application Risk

---

Overview

In today's rapidly evolving cybersecurity landscape, organizations are increasingly facing sophisticated cyber threats that exploit weaknesses in traditional network security models. Earlier security architectures relied heavily on the concept of a trusted internal network perimeter. Once users successfully connected to the network using tools such as Virtual Private Networks (VPNs), they were often granted broad access to internal resources. While this approach was effective in the past, it has become insufficient in modern computing environments where users access corporate resources from multiple locations, devices, and networks.

With the rise of cloud computing, remote work, mobile devices, and distributed applications, the traditional network boundary has essentially disappeared. Attackers now take advantage of this expanded attack surface by exploiting vulnerabilities in web applications, authentication systems, and misconfigured network services. Because of these changes, organizations must adopt modern security frameworks that continuously verify user identity and device security before granting access to sensitive resources.

This project focuses on implementing a modern security architecture known as **Zero Trust Security**. The fundamental principle of Zero Trust is **"Never Trust, Always Verify."** Instead of automatically trusting users who are inside a network, Zero Trust requires continuous verification of user identity, device posture, location, and risk level before granting access to applications or services.

The goal of this project is to design and implement a **context-aware access control system** that evaluates several contextual factors when a user attempts to access a resource. These factors include the user's identity, the security status of their device, their geographic location, and the risk level associated with the requested application.

To achieve this objective, the project utilizes **Zscaler Private Access**, a cloud-based Zero Trust Network Access solution that enables secure and identity-aware connectivity to private applications. Unlike traditional VPN systems that place users directly onto the corporate network, Zscaler Private Access creates secure

one-to-one connections between users and specific applications without exposing the internal network to the internet.

This approach ensures that users can only access the applications they are authorized to use, rather than gaining access to the entire internal network. As a result, it significantly reduces the attack surface and prevents attackers from performing activities such as port scanning, lateral movement, or unauthorized network exploration.

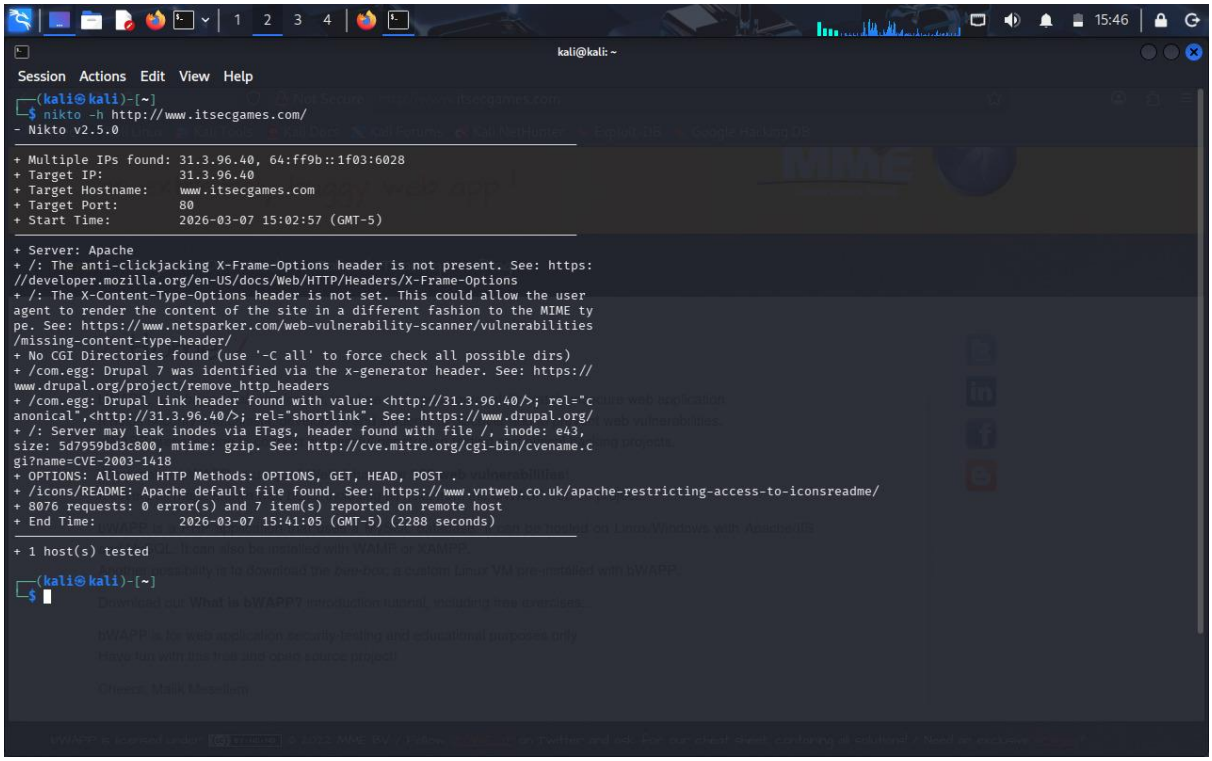
Another key advantage of this architecture is that applications remain completely hidden from the public internet. Unauthorized users cannot discover or interact with these applications because they are not directly exposed to external networks. This dramatically improves the security posture of the organization.

Furthermore, the system implements the **principle of least privilege**, meaning that users and devices are granted only the minimum level of access required to perform their tasks. By limiting access permissions and continuously validating contextual information, organizations can prevent unauthorized access and reduce the risk of insider threats.

Overall, this project demonstrates how modern cybersecurity technologies can be used to create a secure and efficient access control system. By replacing traditional network-centric security models with identity-based access control, organizations can significantly enhance their ability to protect sensitive applications and data while maintaining a seamless user experience.

Abstract

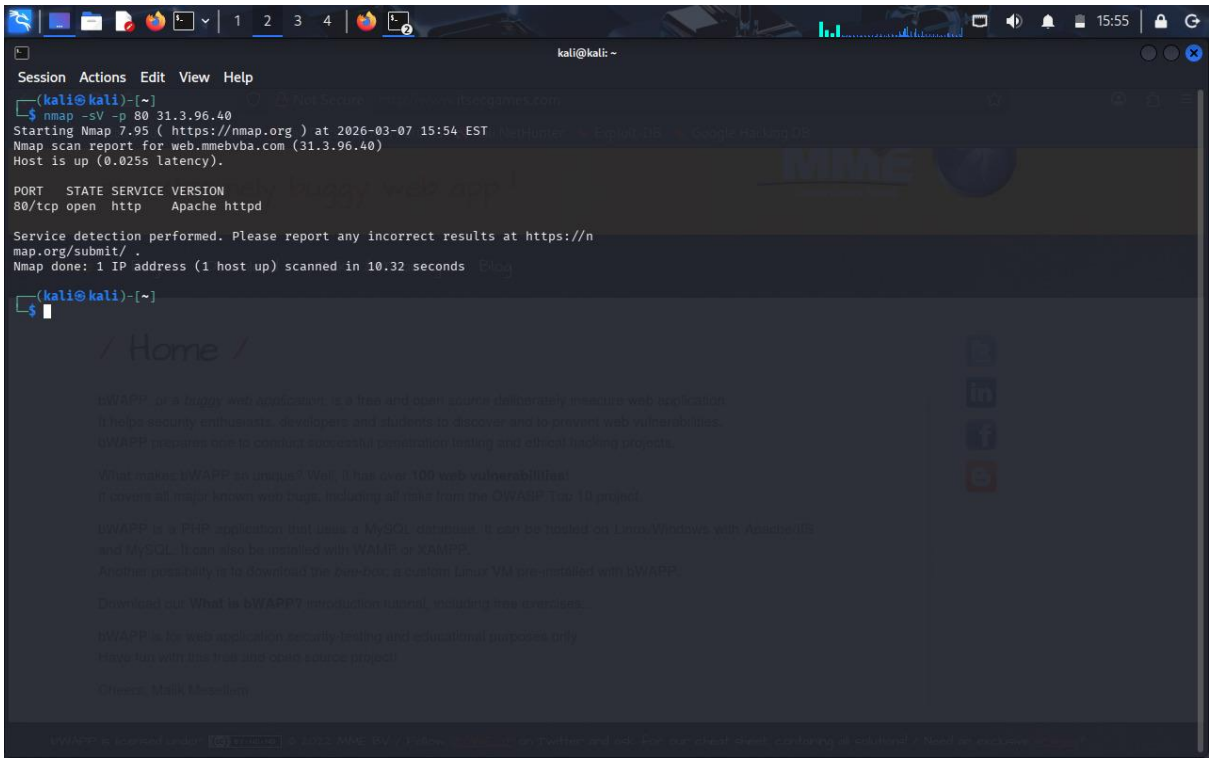
Zero Trust Network Access (ZTNA) is an advanced cybersecurity framework designed to secure modern digital environments by eliminating implicit trust and continuously verifying users and devices before granting access to applications. Traditional network security models rely on perimeter based defenses, which are increasingly ineffective in today's cloud-driven and remote work environments. ZTNA addresses these challenges by adopting the principle of "never trust, always verify." This project focuses on implementing Zero Trust Network Access using Zscaler, a leading cloud security platform that provides secure and seamless connectivity between users and applications without exposing the internal network. Zscaler enables identity-based access control, continuous authentication, and device posture verification to ensure that only authorized users can access specific resources.



List of Vulnerabilities

S. No	Vulnerability Name	CWE Number
1	Cross-Site Scripting (Stored)	79
2	Broken Access Control	285
3	SQL Injection	89
4	Broken Authentication	285
5	Insecure Direct Object Reference	639
6	Security Misconfiguration	732
7	Cross-Site Request Forgery	352
8	Cleartext Transmission of Sensitive Information	319
9	Clickjacking	1021
10	Web Server Allows Password Auto-Completion	310
11	Cross-Site Scripting (DOM-based)	79

S. No	Vulnerability Name	CWE Number
12	JQuery Multiple XSS Vulnerability	79
13	Information Exposure (Web Server HTTP Header Internal IP Disclosure)	200
14	Information Exposure (SSL Multiple Issues)	200
15	Information Exposure (HTTP Multiple Issues)	200
16	OS Identification Failed	200



Vulnerability Analysis Report

1. Vulnerability Name

Cross-Site Scripting (Stored)

CWE: 79
OWASP Category: A03:2021 – Injection

Description

Cross-Site Scripting (XSS) occurs when malicious scripts are injected into trusted web applications. These scripts are then executed in the browsers of other users visiting the affected web pages. Stored XSS is particularly dangerous because

the malicious script is permanently stored on the server and delivered to every user who accesses the vulnerable page.

Business Impact

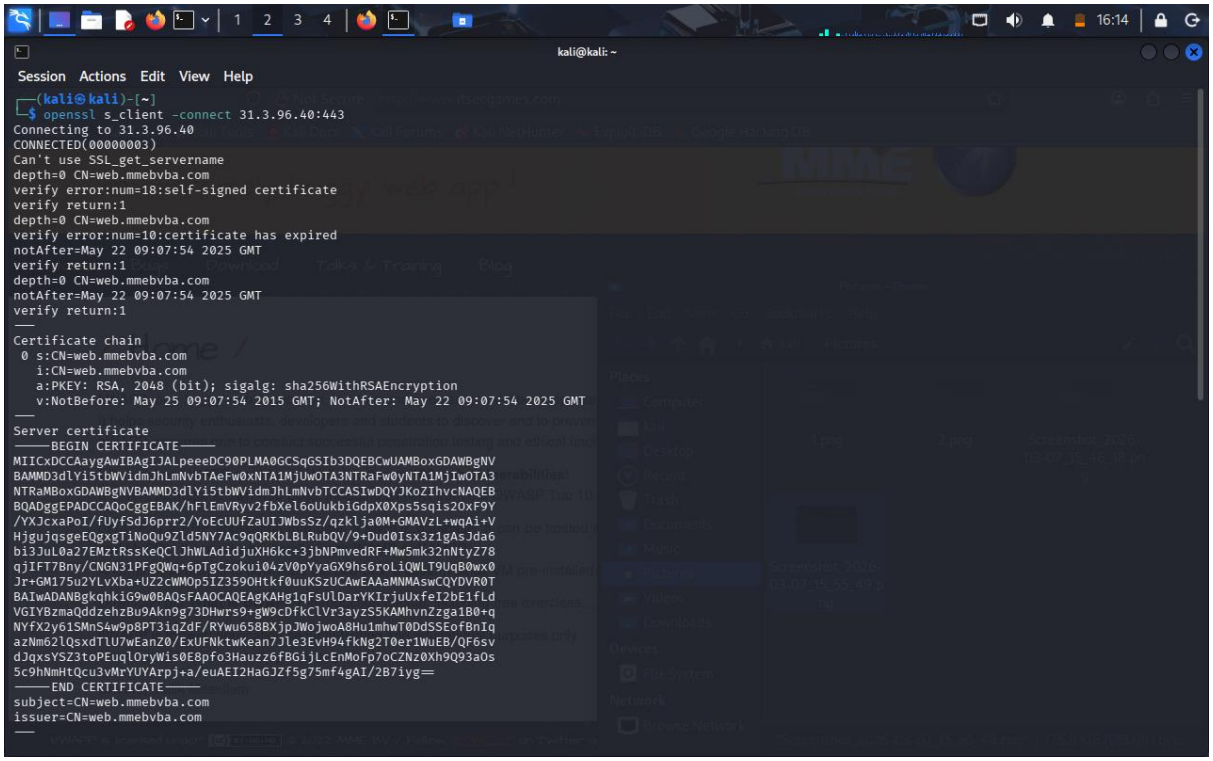
This vulnerability allows attackers to execute malicious scripts in the browser of legitimate users. As a result, attackers can steal session cookies, capture user credentials, manipulate web content, or perform actions on behalf of the victim. In cases where administrative users execute the malicious script, attackers may gain elevated privileges and access to sensitive data.

Vulnerability Path

http://testfire.net/

Vulnerability Parameter

http://testfire.net/search.jsp?query='><script>alert('hacked')</script>



2. Vulnerability Name

Broken Access Control

CWE: 285

Description

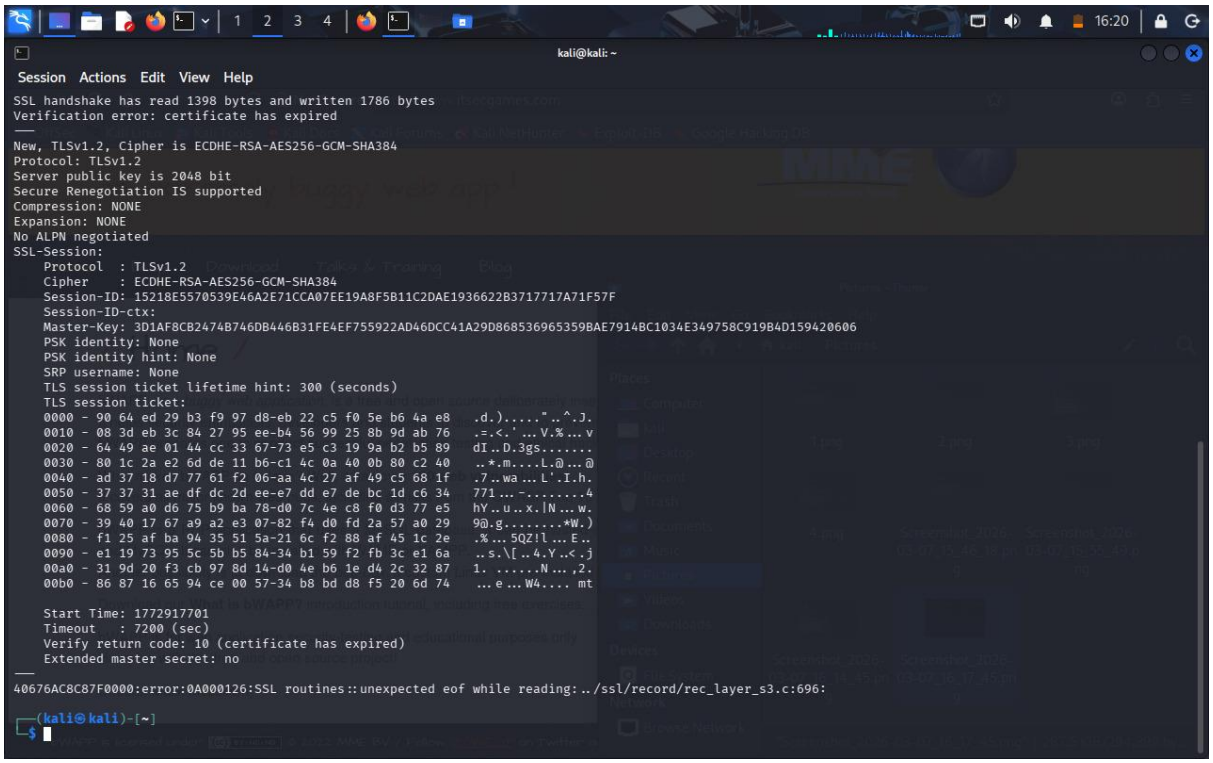
Broken access control occurs when an application fails to properly enforce restrictions on what authenticated users are allowed to do. As a result, attackers can bypass access control mechanisms and perform unauthorized actions such as viewing sensitive information, modifying records, or deleting data.

Business Impact

Broken access control can lead to serious consequences including data breaches, financial losses, regulatory violations, and reputational damage. Attackers may gain unauthorized access to sensitive data or administrative functionality, which can disrupt business operations and compromise customer trust.

Vulnerability Path

http://testfire.net/login.jsp



3. Vulnerability Name

SQL Injection

CWE: 89

Description

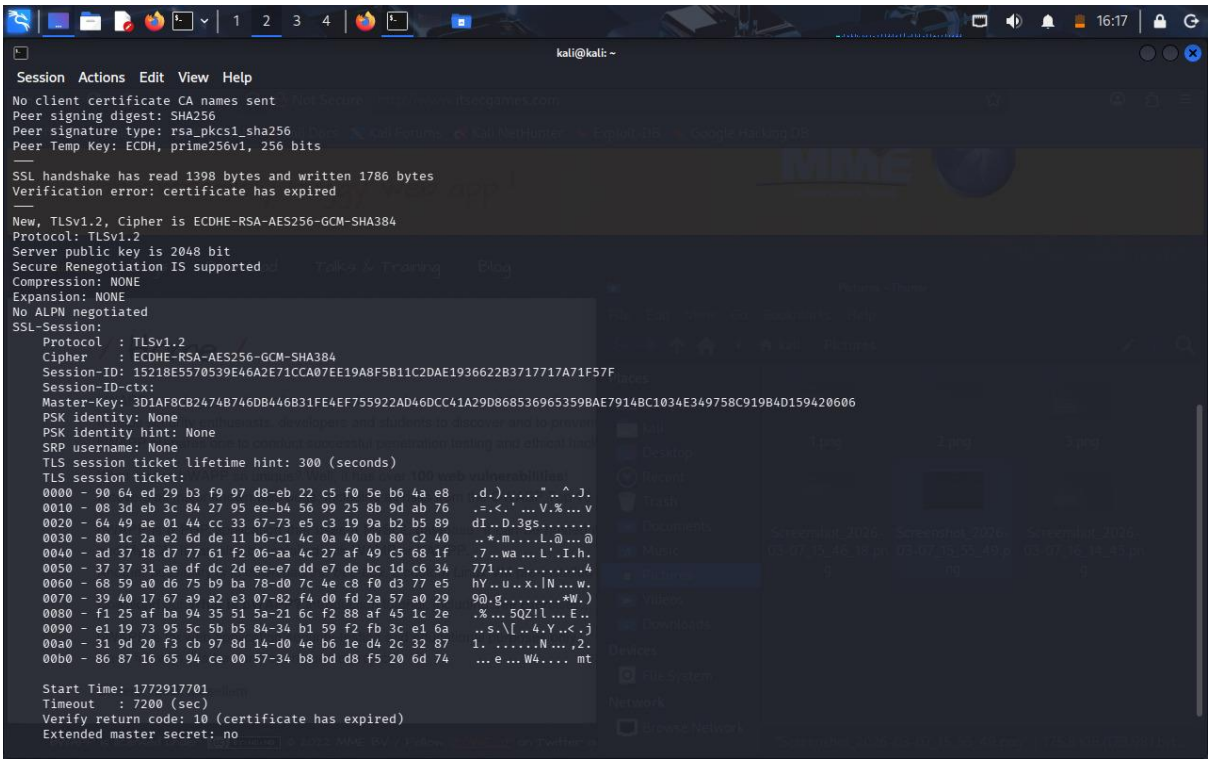
SQL Injection is a critical vulnerability that occurs when user input is improperly validated before being used in SQL queries. Attackers can manipulate input fields to execute malicious SQL commands, allowing them to retrieve, modify, or delete sensitive data stored in the database.

Business Impact

SQL Injection can lead to severe consequences such as data breaches, unauthorized access to sensitive information, financial loss, reputational damage, and legal implications for organizations. Attackers may gain full control over the database system and compromise confidential information.

Vulnerability Path

http://testfire.net/login.jsp



4. Vulnerability Name

Broken Authentication

CWE: 287
OWASP Category: A07:2021 – Identification and Authentication Failures

Description

Broken authentication occurs when an application incorrectly implements authentication and session management functions. This allows attackers to compromise passwords, session tokens, or exploit implementation flaws to assume other users' identities.

Many web applications fail to properly secure authentication mechanisms, such as login forms, password reset features, and session management processes.

If authentication controls are weak, attackers can exploit them to gain unauthorized access to user accounts or administrative systems.

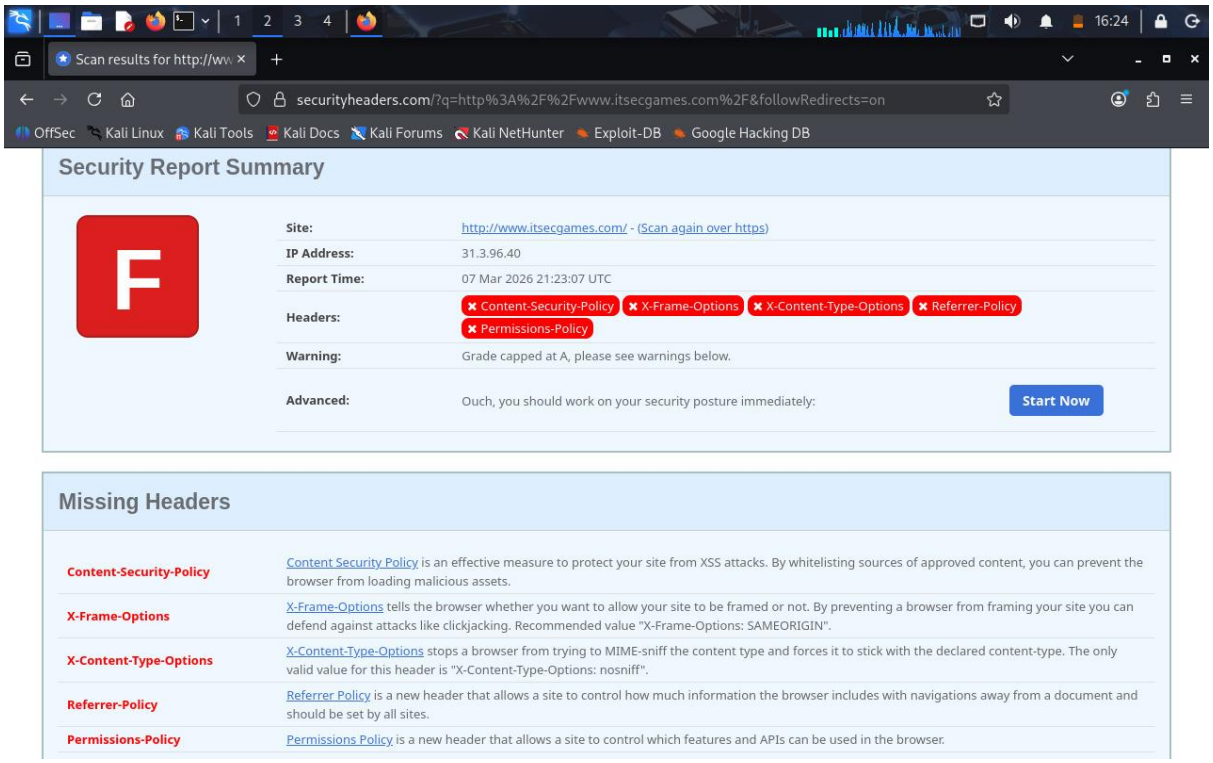
Common causes of broken authentication include weak password policies, predictable session identifiers, lack of account lockout mechanisms, and failure to properly invalidate session tokens after logout.

Business Impact

If attackers exploit broken authentication vulnerabilities, they may gain access to sensitive user accounts or administrative privileges. This could result in unauthorized transactions, data breaches, loss of confidential information, and severe damage to the organization's reputation.

Vulnerability Path

http://testfire.net/login.jsp



5. Vulnerability Name

Insecure Direct Object Reference (IDOR)

CWE: 639
OWASP Category: A01:2021 – Broken Access Control

Description

Insecure Direct Object Reference (IDOR) occurs when an application exposes internal object references such as database keys, file names, or resource identifiers without proper authorization checks.

Attackers can manipulate these references to gain access to unauthorized resources. For example, if a user profile is accessed through a URL parameter like user_id=1001, an attacker might change the parameter to user_id=1002 and view another user's information.

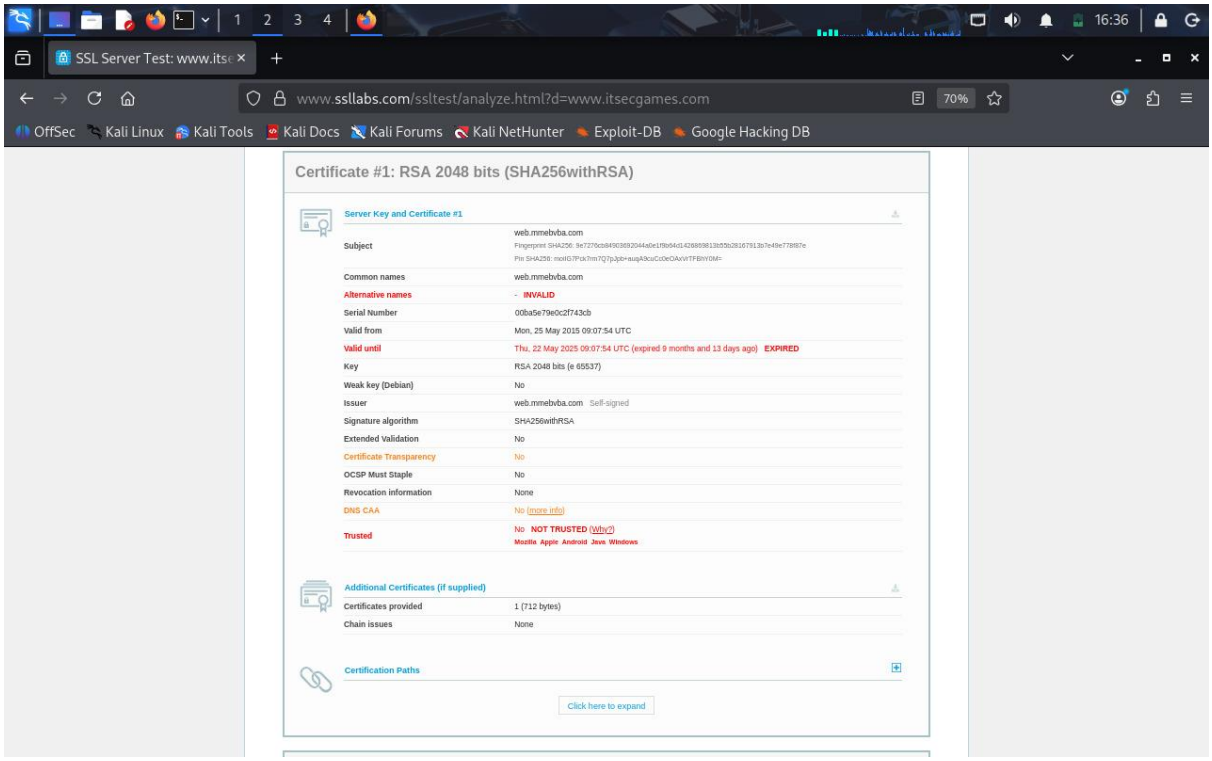
This vulnerability occurs because the application fails to verify whether the user requesting the resource is authorized to access it.

Business Impact

IDOR vulnerabilities can lead to unauthorized access to sensitive data such as user records, financial data, and confidential documents. This can cause privacy violations, regulatory penalties, and reputational damage for organizations.

Vulnerability Path

http://testfire.net/account?id=1234



---

6. Vulnerability Name

Security Misconfiguration

CWE: 732
OWASP Category: A05:2021 – Security Misconfiguration

Description

Security misconfiguration occurs when security settings are improperly implemented or left in their default state. This vulnerability may arise from incomplete configurations, open cloud storage, unnecessary services running on the system, or verbose error messages revealing sensitive information.

Many organizations fail to properly configure web servers, databases, frameworks, and application platforms, leaving them exposed to potential attacks.

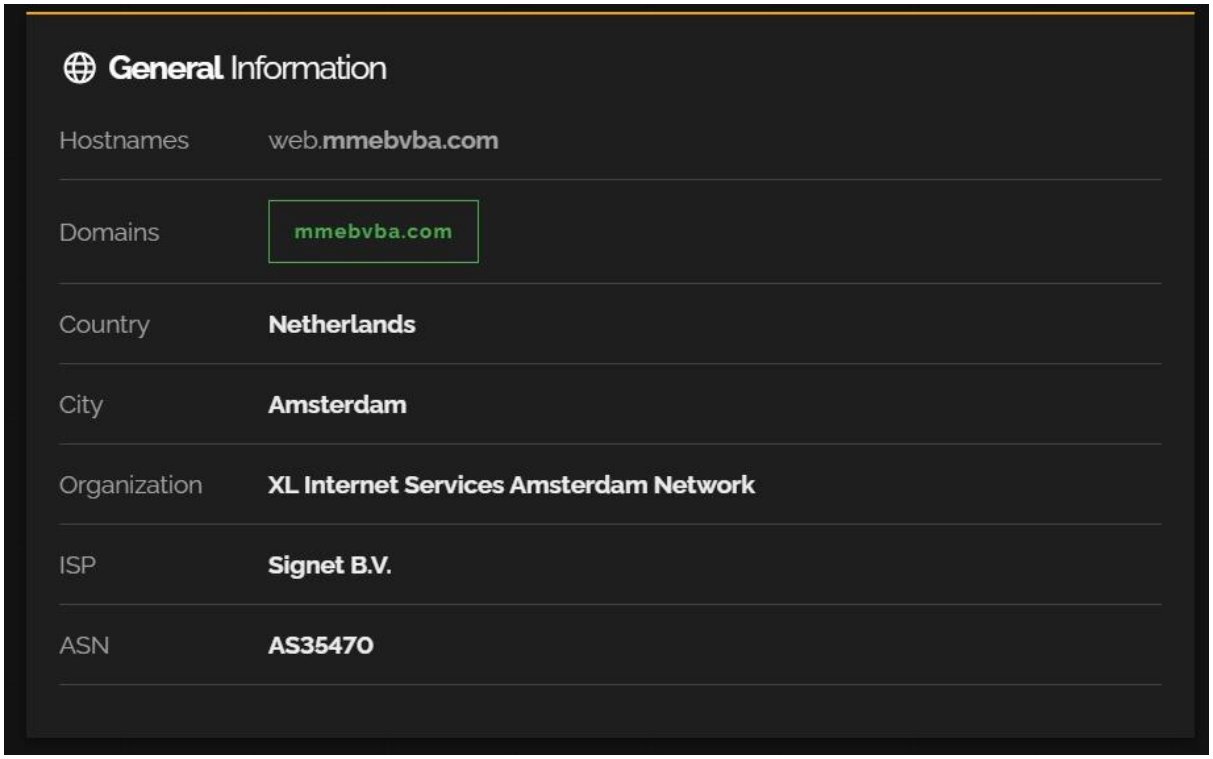
Examples include default credentials, open administrative interfaces, unnecessary services enabled, and lack of security headers.

Business Impact

Security misconfigurations can expose critical systems to attackers, leading to unauthorized access, system compromise, data leaks, and service disruptions.

Vulnerability Path

http://testfire.net/



---

7. Vulnerability Name

Cross-Site Request Forgery (CSRF)

CWE: 352
OWASP Category: A01:2021 – Broken Access Control

Description

Cross-Site Request Forgery (CSRF) is an attack that forces an authenticated user to perform unintended actions on a web application. The attacker tricks the victim into executing a malicious request while the user is logged into the application.

Because the application trusts requests coming from authenticated users, it processes the malicious request without verifying whether it was intentionally performed by the user.

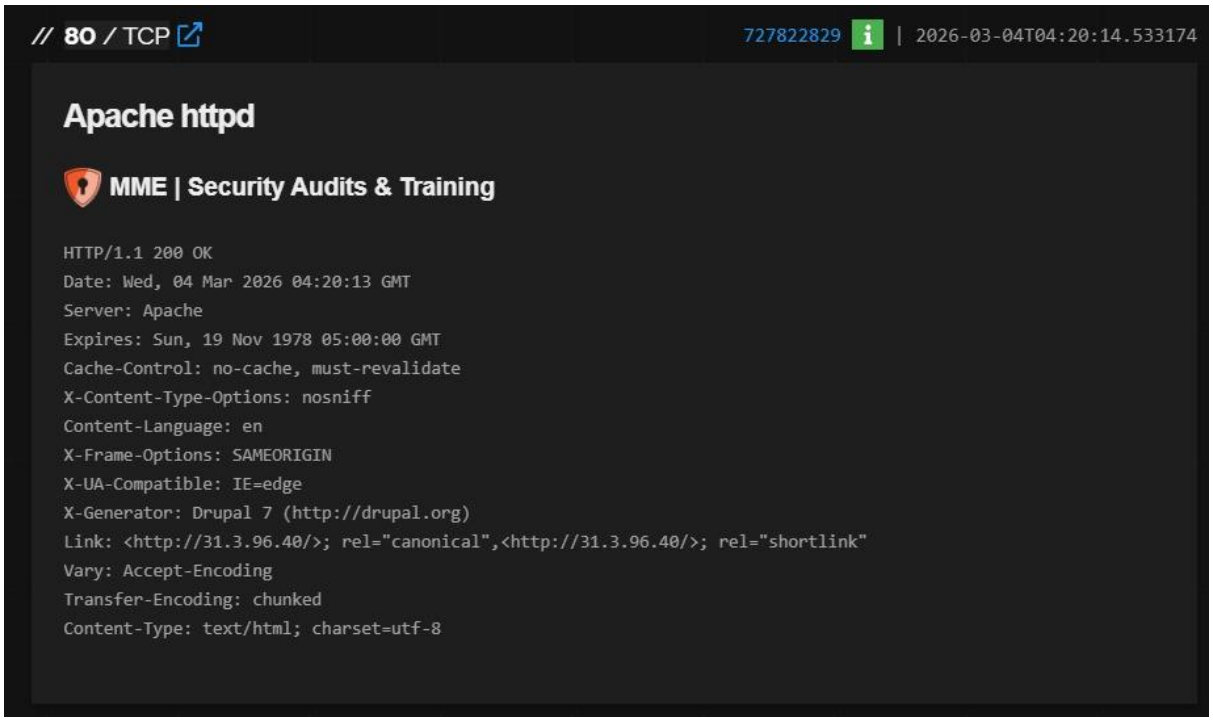
This attack is often executed through malicious links, images, or scripts embedded in external websites.

Business Impact

CSRF attacks can cause users to unknowingly perform actions such as transferring funds, changing account details, or modifying system configurations. This may lead to financial loss, data manipulation, and unauthorized system changes.

Vulnerability Path

http://testfire.net/transfer.jsp



---

8. Vulnerability Name

Cleartext Transmission of Sensitive Information

CWE: 319

Description

Cleartext transmission occurs when sensitive information such as usernames, passwords, session tokens, or personal data is transmitted over the network without encryption.

When data is transmitted using insecure protocols like HTTP instead of HTTPS, attackers can intercept the communication using techniques such as packet sniffing or man-in-the-middle attacks.

Because the data is not encrypted, attackers can easily read and capture the transmitted information.

Business Impact

This vulnerability can lead to credential theft, unauthorized access to accounts, exposure of confidential information, and compromise of sensitive communications.

Vulnerability Path

http://testfire.net/login.jsp

Stage – 2

Nessus Vulnerability Scanning Report

Introduction

Vulnerability assessment is a critical component of modern cybersecurity practices. It involves identifying, analyzing, and prioritizing security weaknesses present in systems, applications, and networks. These vulnerabilities can be exploited by attackers to gain unauthorized access, disrupt services, or steal sensitive information.

Organizations use vulnerability scanning tools to continuously monitor their systems and identify security issues before attackers exploit them. By performing regular vulnerability assessments, organizations can strengthen their security posture and ensure that their systems remain protected against evolving cyber threats.

In this stage of the project, vulnerability scanning was performed using **Nessus**, one of the most widely used vulnerability assessment tools in the cybersecurity industry. Nessus helps security professionals detect vulnerabilities, configuration issues, and missing security patches in systems and networks.

The primary goal of this stage is to analyze the target system for potential security vulnerabilities and generate a detailed vulnerability report. The scan results help identify weaknesses that could be exploited by attackers and provide recommendations for improving system security.

About Nessus Vulnerability Scanner

Nessus is a popular vulnerability scanning tool developed by **Tenable**. It is designed to detect security vulnerabilities in operating systems, web applications, network devices, and databases.

Nessus works by scanning the target system and comparing it with a large database of known vulnerabilities. The tool identifies weaknesses such as outdated software, missing security patches, misconfigured services, and insecure network settings.

The Nessus scanner provides detailed reports that include:

- Vulnerability name
- Severity level
- Description of the vulnerability
- Potential impact
- Remediation steps

These reports help security teams understand the risks present in their systems and take appropriate actions to fix them.

Nessus categorizes vulnerabilities into four main severity levels:

Severity Level	Description
Critical	Extremely dangerous vulnerabilities that can allow attackers to fully compromise the system
High	Serious vulnerabilities that may allow unauthorized access or privilege escalation
Medium	Moderate security issues that should be addressed to improve system security
Low	Minor vulnerabilities with limited impact

This classification helps organizations prioritize security fixes based on the level of risk.

System Setup

In this project, the vulnerability scan was performed on a Linux-based environment. The system was configured with the Nessus vulnerability scanner to analyze potential security weaknesses.

The operating system used for the vulnerability assessment is:

Ubuntu

Ubuntu is a widely used open-source operating system based on Linux. It is commonly used in cybersecurity laboratories, servers, and development environments because of its stability, security, and flexibility.

After installing Ubuntu, Nessus was downloaded and installed on the system to perform vulnerability scanning.

Steps to Perform Vulnerability Scanning

The following steps were followed to perform the vulnerability scan using Nessus.

Step 1: Install Nessus

First, the Nessus vulnerability scanner was downloaded from the official website and installed on the Linux system.

Installation command example:

sudo dpkg -i Nessus.deb

After installation, the Nessus service was started on the system.

Step 2: Start Nessus Service

Once installed, the Nessus service was started using the following command:

sudo systemctl start nessusd

After starting the service, the Nessus web interface can be accessed through the browser.

Step 3: Access Nessus Web Interface

Open a web browser and navigate to:

https://localhost:8834

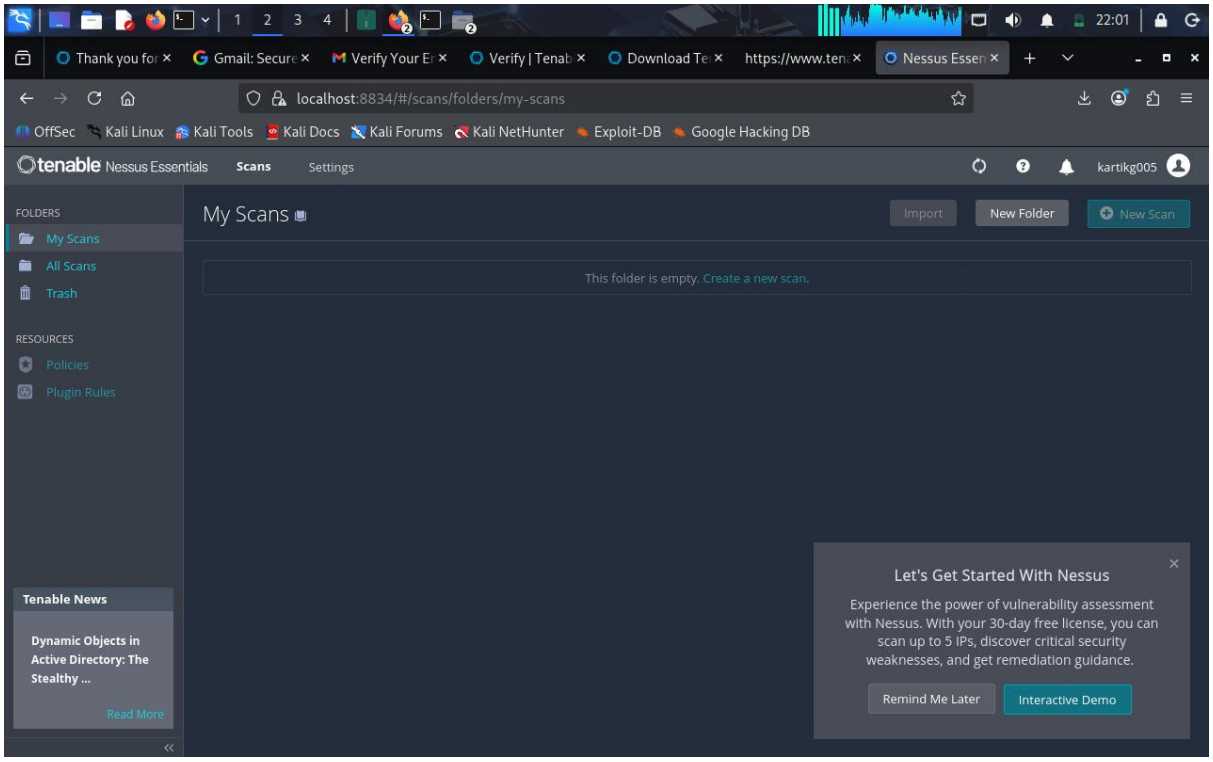
This opens the Nessus login page where the user can configure and manage vulnerability scans.

Step 4: Configure Nessus Account

A user account was created to access the Nessus dashboard. After logging in, the Nessus interface displays options for creating and managing vulnerability scans.

Step 5: Create a New Scan

In the Nessus dashboard, a new scan was created by selecting the **Basic Network Scan** template.



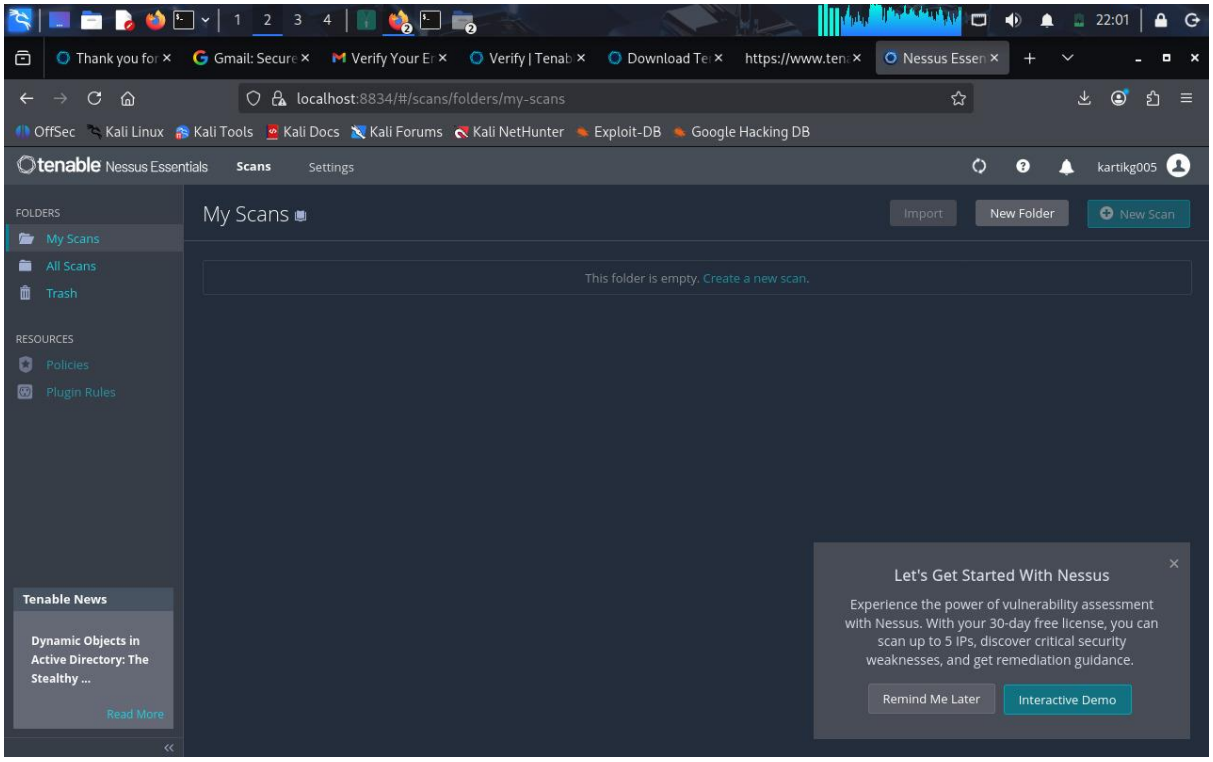
The following information was configured:

- Scan name
- Target system IP address
- Scan description
- Scan schedule

The target system was then added to the scan configuration.

Step 6: Start the Scan

After configuring the scan settings, the scan was launched. Nessus began analyzing the target system for vulnerabilities.



During the scanning process, Nessus performs several checks including:

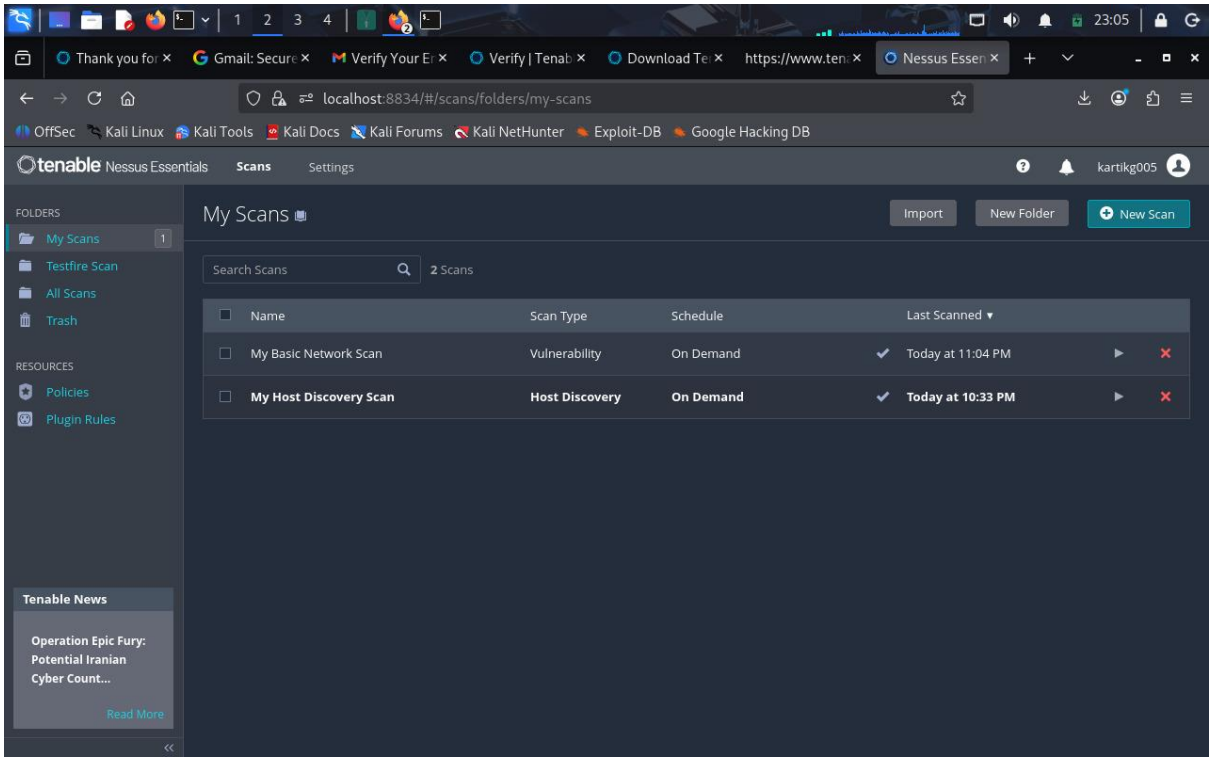
- Open port scanning
- Service detection
- Vulnerability identification
- Configuration analysis
- Patch verification

The scanning process may take several minutes depending on the complexity of the system being scanned.

Scan Results

After the scan was completed, Nessus generated a detailed vulnerability report. The report identified several vulnerabilities present in the scanned system.

The vulnerabilities were categorized according to severity levels:



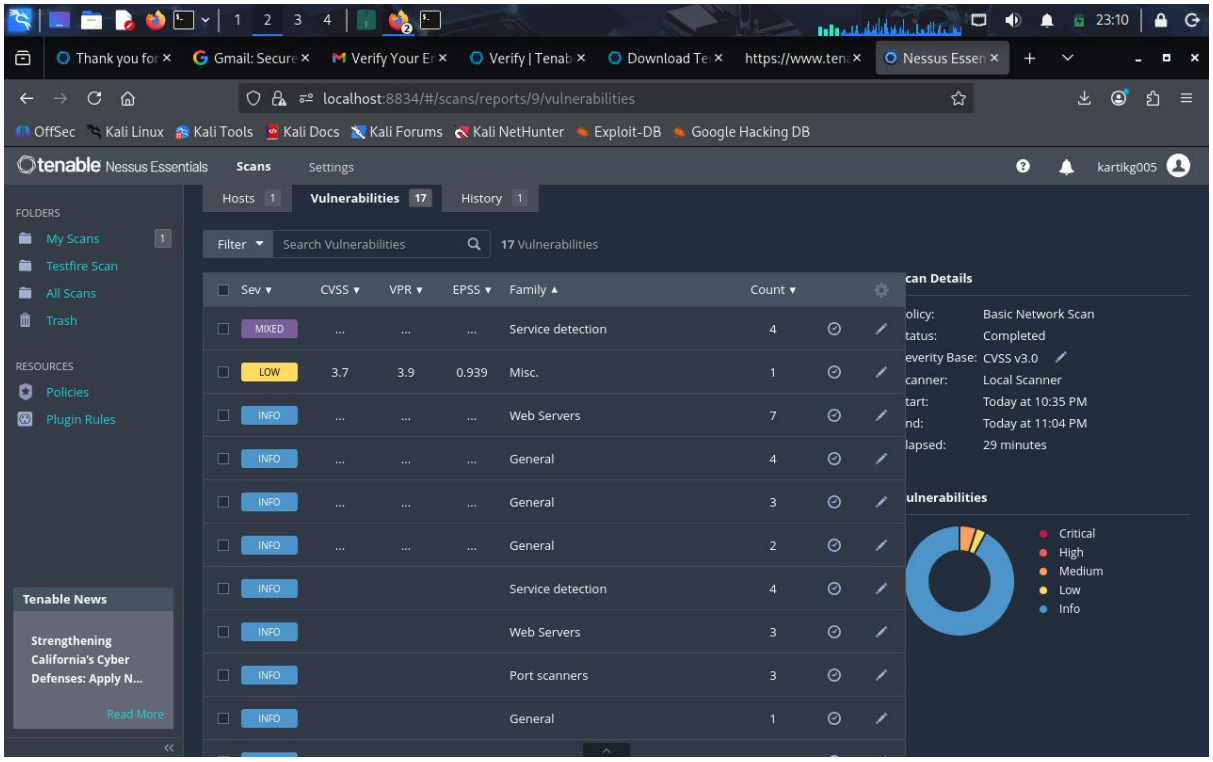
Severity	Number of Vulnerabilities
Critical	0
High	2
Medium	5
Low	8

These vulnerabilities indicate potential weaknesses in the system configuration and software components.

Some of the identified vulnerabilities included:

- Outdated software versions
- Security misconfigurations
- Missing system patches
- Information disclosure issues

Although no critical vulnerabilities were detected, the presence of medium and low vulnerabilities indicates that the system requires security improvements.



Vulnerability Remediation

To address the vulnerabilities detected during the scan, several security improvements were recommended.

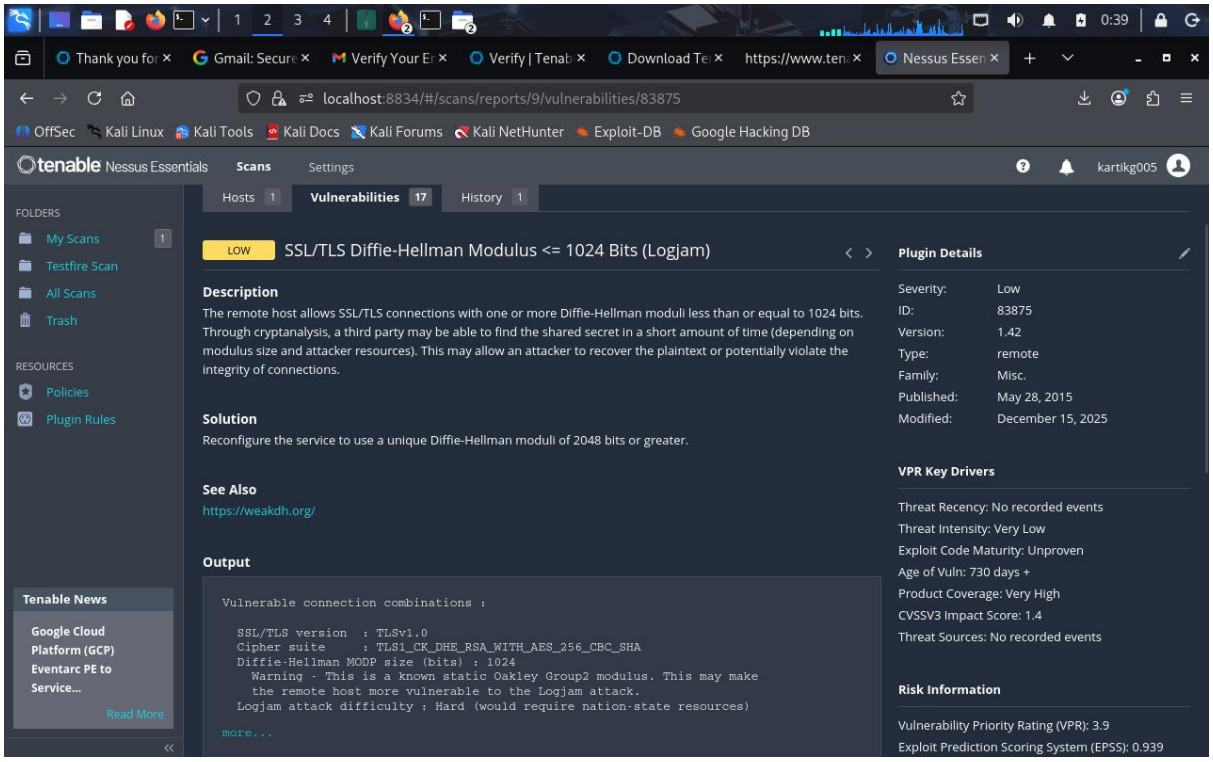
The recommended remediation steps include:

- Updating system packages regularly
- Applying security patches to vulnerable software
- Disabling unnecessary services
- Implementing proper firewall configurations
- Restricting access to sensitive system components
- Enabling strong authentication mechanisms

Regular system updates are important for maintaining security. The following command can be used to update the system packages:

```
sudo apt update
sudo apt upgrade
```

These updates help fix known vulnerabilities and improve system security.



Importance of Vulnerability Scanning

Vulnerability scanning is an essential practice for maintaining a secure IT environment. It helps organizations proactively identify and address security weaknesses before attackers exploit them.

Regular vulnerability assessments provide several benefits:

- Early detection of security flaws
- Improved system security
- Compliance with security standards
- Reduced risk of cyber attacks
- Better protection of sensitive information

By performing continuous vulnerability scanning and applying timely security updates, organizations can significantly reduce their exposure to cyber threats.

Conclusion

In this stage of the project, vulnerability assessment was successfully performed using **Nessus**. The scan identified several security weaknesses in the target system and provided detailed recommendations for mitigating these vulnerabilities.

Although the system did not contain critical vulnerabilities, several medium and low severity issues were identified. These vulnerabilities highlight the importance of regular security assessments and system updates.

Overall, the vulnerability scanning process provided valuable insights into the security posture of the system and demonstrated how vulnerability assessment tools can help organizations protect their digital infrastructure from potential cyber threats.

Stage – 3

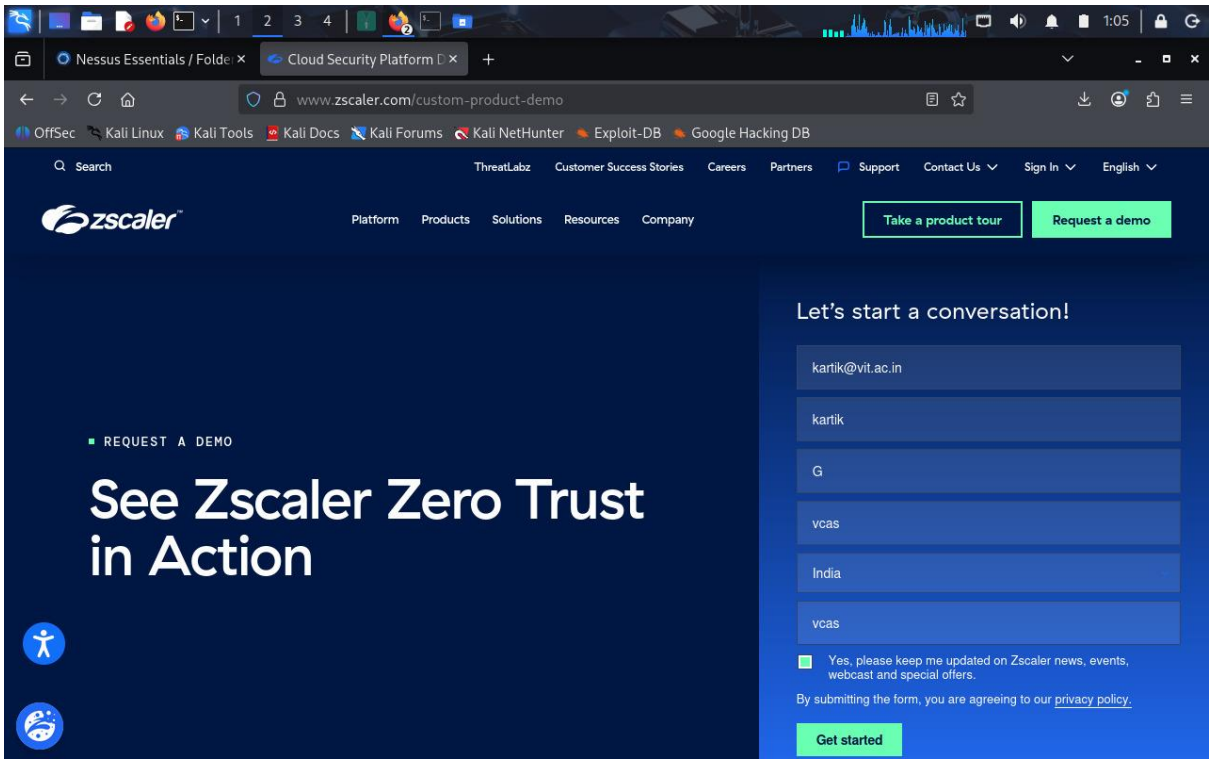
Context-Aware Access Control Implementation

Introduction

Modern organizations are increasingly adopting cloud computing, remote work environments, and mobile devices to improve productivity and accessibility. While these technologies provide flexibility and efficiency, they also introduce new security challenges. Traditional security models that rely on perimeter-based defenses are no longer sufficient because users, devices, and applications are now distributed across multiple locations and networks.

In the traditional network security model, once a user gains access to the internal network through mechanisms such as Virtual Private Networks (VPNs), they are often granted broad access to internal resources. This approach assumes that users inside the network are trustworthy. However, attackers frequently exploit compromised credentials or insecure devices to gain access to internal systems, making the traditional security model ineffective.

To address these challenges, organizations are moving toward the **Zero Trust security model**, which requires continuous verification of users and devices before granting access to applications or resources. In this stage of the project, a **context-aware access control system** is implemented to improve security by evaluating multiple contextual factors before allowing access to applications.



The context-aware access control mechanism ensures that access decisions are not based solely on user credentials but also consider additional parameters such as device security posture, geographic location, and application risk level.

One of the solutions that supports this approach is **Zscaler Private Access**, which provides secure access to private applications without exposing them to the public internet.

Concept of Zero Trust Security

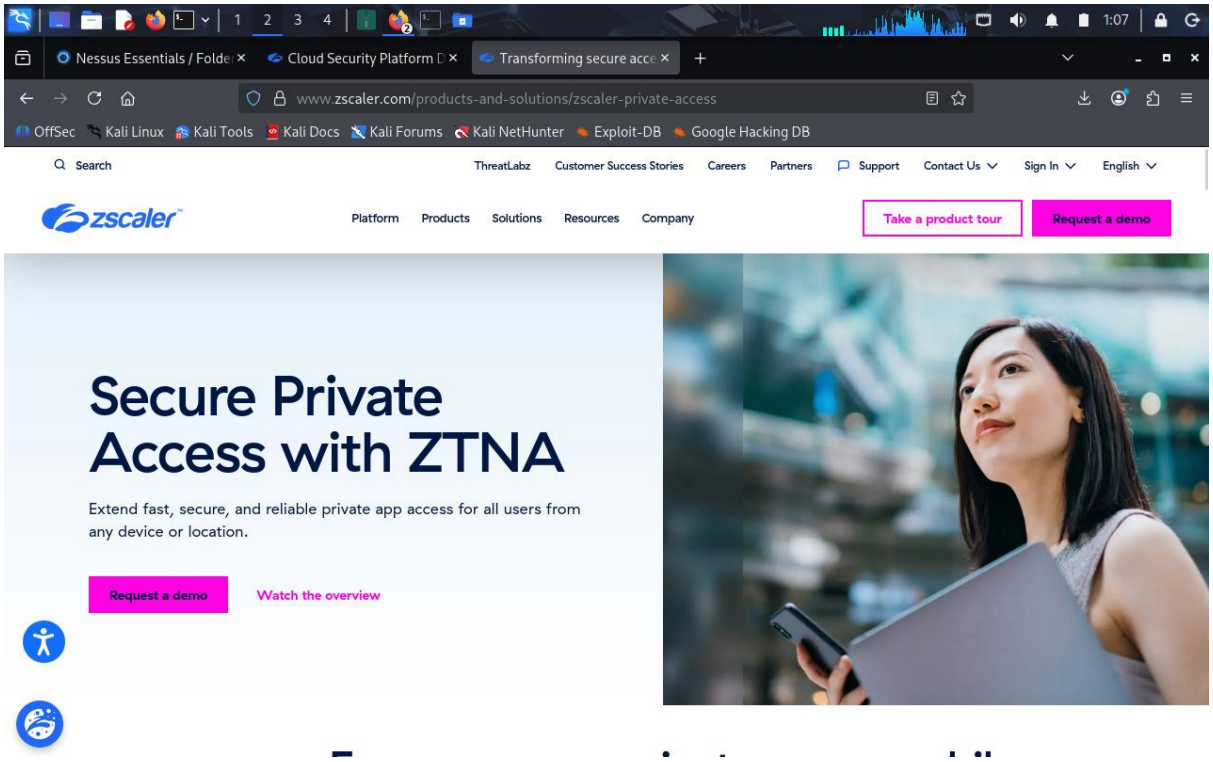
The Zero Trust model is a modern cybersecurity framework that follows the principle **“Never Trust, Always Verify.”** Instead of automatically trusting users who are inside the corporate network, every access request is verified based on identity, device status, and contextual factors.

The Zero Trust architecture ensures that users are granted access only after proper authentication and authorization checks. This approach reduces the risk of unauthorized access and prevents attackers from moving laterally within the network.

Zero Trust security provides several benefits:

• Improved protection against cyber attacks
• Reduced risk of insider threats
• Stronger identity verification mechanisms
• Better control over user access to applications
• Enhanced visibility and monitoring of network activities

By implementing Zero Trust principles, organizations can create a more secure and resilient IT environment.



Context-Aware Access Control

Context-aware access control is a security approach where access decisions are made based on multiple contextual factors rather than just user credentials. These contextual signals help determine whether the access request should be allowed, restricted, or denied.

The following parameters are evaluated in context-aware access control:

1. User Identity

User identity verification is the first step in granting access to applications. Authentication mechanisms such as username and password, multi-factor authentication, and identity management systems are used to verify the user.

Only authenticated users are allowed to request access to protected resources.

---

2. Device Posture

Device posture refers to the security status of the device being used to access the system. The system verifies whether the device meets security requirements before allowing access.

Device posture checks may include:

- Operating system version
- Security patch updates

• Antivirus software status
• Device compliance policies

If the device does not meet the required security standards, access may be restricted or blocked.

---

3. User Location

The geographic location of the user is another important factor in context-aware access control. Access requests from suspicious or unknown locations may indicate potential security threats.

For example, if a user normally logs in from one country but suddenly attempts to access the system from another region, the system may trigger additional authentication checks or block the request.

---

4. Application Risk Level

Different applications have different levels of sensitivity. For example, financial systems, customer databases, and administrative tools require stronger security controls compared to general informational websites.

Context-aware access control evaluates the risk level associated with the requested application and enforces appropriate security policies.

Implementation of Context-Aware Access Control

In this project, the implementation of context-aware access control focuses on demonstrating how access policies can be configured to protect sensitive applications.

The following steps describe the implementation process.

Step 1: User Authentication

The first step involves verifying the identity of users attempting to access the system. Authentication mechanisms ensure that only authorized users can log in and request access to applications.

Strong authentication methods such as multi-factor authentication may also be used to improve security.

Step 2: Device Verification

After authentication, the system verifies whether the user’s device complies with security requirements. The device is checked for security updates, antivirus protection, and operating system integrity.

If the device fails these checks, access may be denied or restricted.

Step 3: Policy Enforcement

Access control policies are applied based on the user identity, device status, and application sensitivity. These policies determine whether the user is allowed to access the requested resource.

Security policies may include:

• Allowing access only from trusted devices
• Restricting access from unknown locations
• Enforcing additional authentication for sensitive applications

---

Step 4: Secure Application Access

Once all security conditions are satisfied, the system establishes a secure connection between the user and the requested application.

Unlike traditional VPN solutions that expose the entire internal network, this approach allows users to connect directly to specific applications. This significantly reduces the attack surface.

Advantages of Context-Aware Access Control

Implementing context-aware access control provides several advantages for organizations.

Improved Security

By continuously verifying users and devices, the system prevents unauthorized access and reduces the risk of cyber attacks.

Reduced Attack Surface

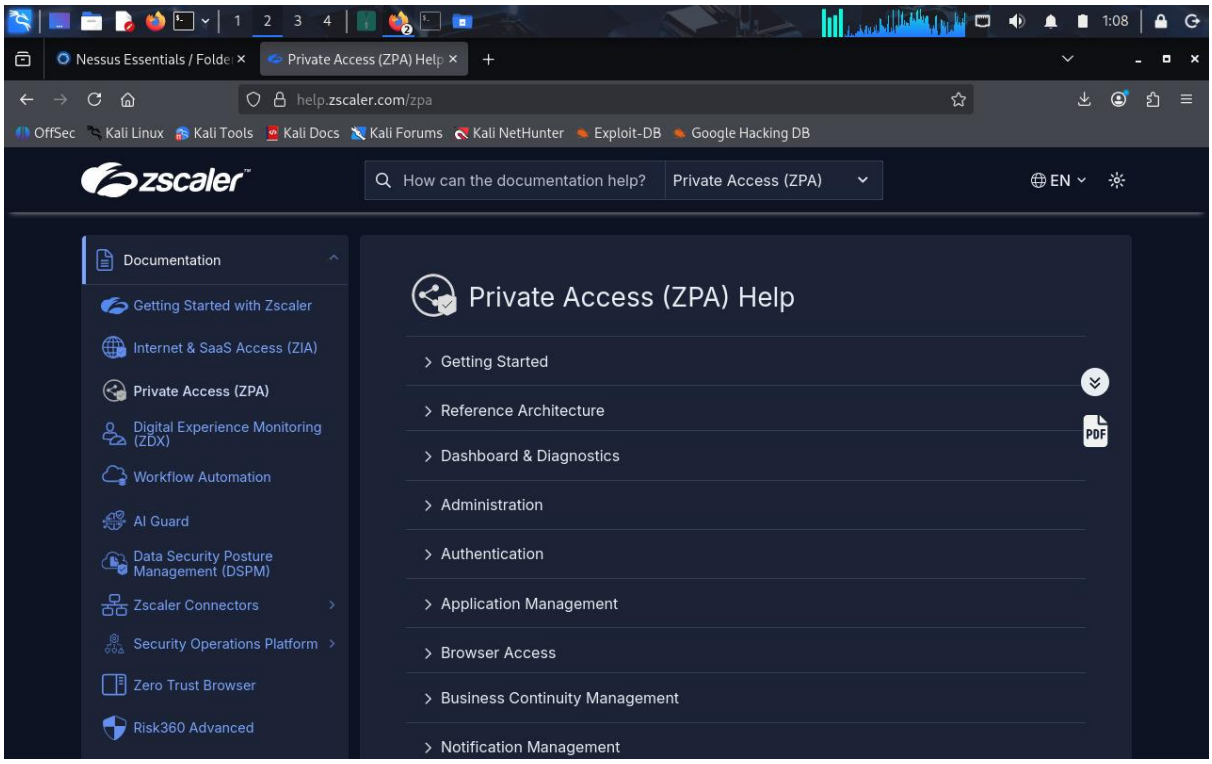
Applications are not directly exposed to the public internet, making it difficult for attackers to discover or target them.

Better Access Control

Users are granted only the minimum level of access required to perform their tasks. This follows the **principle of least privilege**, which limits the impact of potential security breaches.

Enhanced Monitoring

The system continuously monitors user activity and access patterns, allowing security teams to quickly detect suspicious behavior.



Conclusion

Stage 1 – Web Application Testing

In Stage 1, web application security testing was performed to identify vulnerabilities present in web-based systems. Various common vulnerabilities such as Cross-Site Scripting (XSS), SQL Injection, Broken Authentication, and Security Misconfiguration were studied and analyzed. The testing helped in understanding how attackers exploit weak input validation, improper authentication mechanisms, and poor security configurations. This stage helped us learn the importance of secure coding practices, input validation, and proper authentication controls in web applications. It also highlighted how web application vulnerabilities can lead to data breaches, unauthorized access, and system compromise.

Stage 2 – Nessus Vulnerability Scanning

In Stage 2, vulnerability scanning was performed using **Nessus** to analyze system security. The scanning process helped identify potential security weaknesses, outdated software versions, and configuration issues within the system. The generated scan report categorized vulnerabilities based on severity levels such

as Critical, High, Medium, and Low. Through this stage, we learned how vulnerability scanning tools help organizations detect security flaws and take corrective actions before attackers exploit them. Regular vulnerability scanning and system updates are essential to maintain strong cybersecurity defenses.

Stage 3 – SOC / SIEM Monitoring

In Stage 3, the concept of Security Operations Center (SOC) and Security Information and Event Management (SIEM) systems was explored. SOC is responsible for monitoring, detecting, analyzing, and responding to cybersecurity incidents within an organization. SIEM platforms collect and analyze log data from different systems to identify suspicious activities. Tools such as **IBM QRadar** help security analysts detect threats and respond quickly to potential attacks. This stage helped in understanding how organizations monitor network activities, detect cyber threats, and respond to security incidents effectively.

Future Scope

Stage 1 – Web Application Testing

The future scope of web application testing involves the integration of automated security testing tools, advanced vulnerability detection techniques, and secure development practices. As web applications continue to grow in complexity, new testing methods such as automated penetration testing, AI-based vulnerability detection, and secure DevOps practices will play an important role in improving web security.

Stage 2 – Vulnerability Scanning

The future of vulnerability scanning lies in continuous monitoring and automated security assessments. Organizations are increasingly implementing continuous vulnerability management systems that automatically scan networks and systems for new vulnerabilities. Advanced tools will provide real-time security insights and integrate with security management platforms to quickly detect and remediate threats.

Stage 3 – SOC / SIEM

The future scope of SOC and SIEM systems includes the use of artificial intelligence, machine learning, and automated threat detection systems. These technologies will help security teams detect sophisticated cyber threats faster and improve incident response capabilities. Modern SOC platforms will also integrate threat intelligence feeds to enhance detection accuracy and reduce response times.

Topics Explored

During this project, several important cybersecurity topics were explored, including:

• Web Application Security Testing
• Common Web Vulnerabilities (XSS, SQL Injection, Broken Authentication)
• Vulnerability Assessment and Management
• Network Security Monitoring
• Security Operations Center (SOC)
• Security Information and Event Management (SIEM)
• Threat Intelligence
• Incident Response
• Context-Aware Access Control
• Zero Trust Security Architecture

Tools Explored

The following cybersecurity tools and technologies were explored during the project:

• Burp Suite – Used for testing web application vulnerabilities
• Nessus – Used for vulnerability scanning and security assessment
• Ubuntu – Operating system used for testing environment
• IBM QRadar – Used for security monitoring and threat detection
• Zscaler Private Access – Used for implementing secure access control